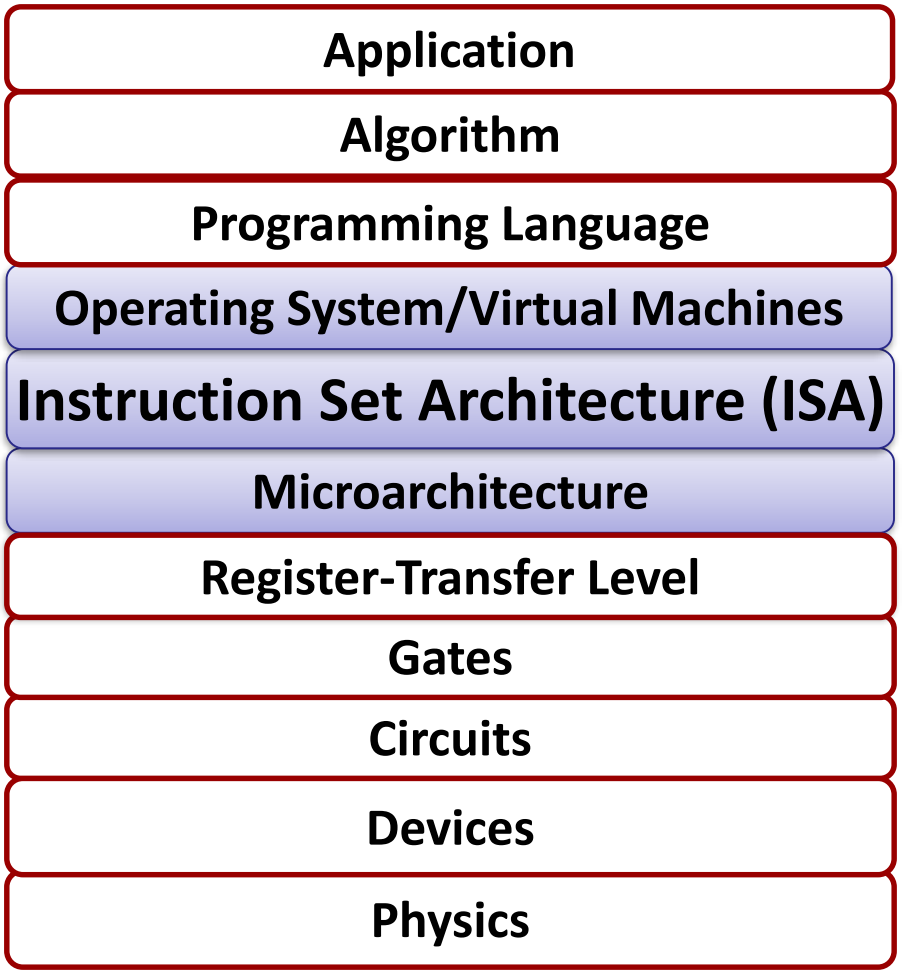
THE UNIVERSITY OF CHICAGO

Machine-Level Programming I: Basics of Instruction Set Architecture

CS144 Fall 2024
Section 3.1-3.5

Yanjing Li

Big Picture



Lecture Goals

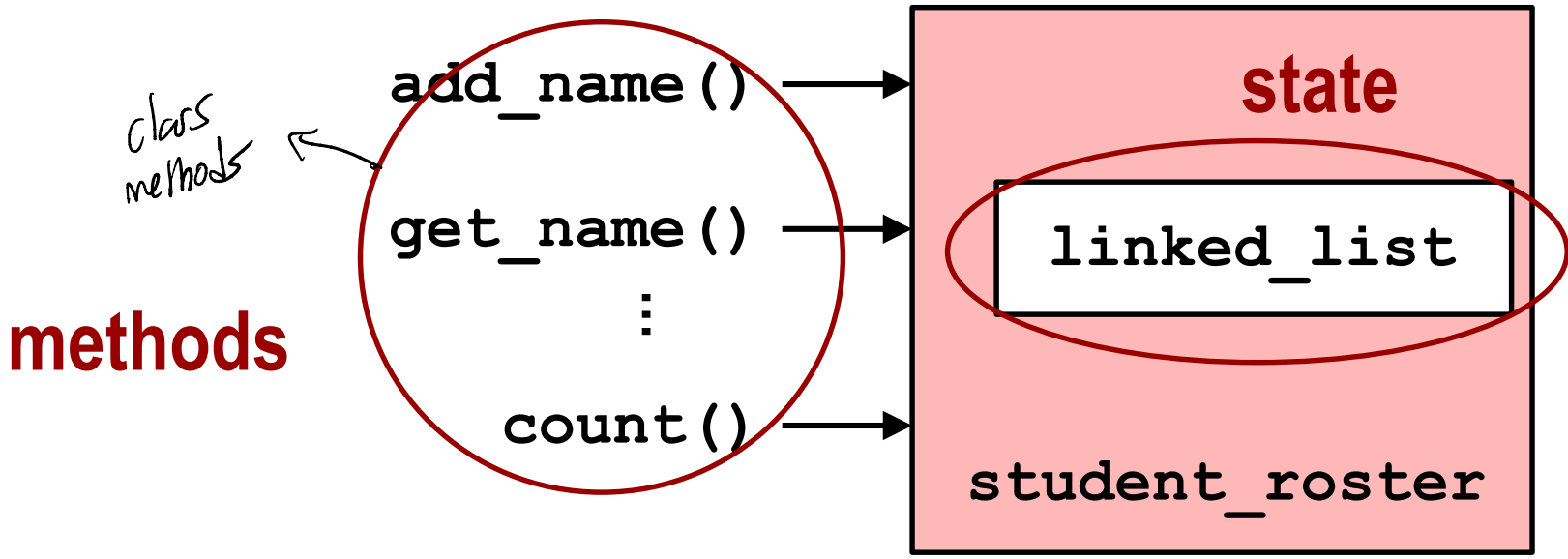
- **Define Instruction Set Architecture (ISA)**
- Basics of memory
- Basics of x86-64 ISA

Object: state + methods

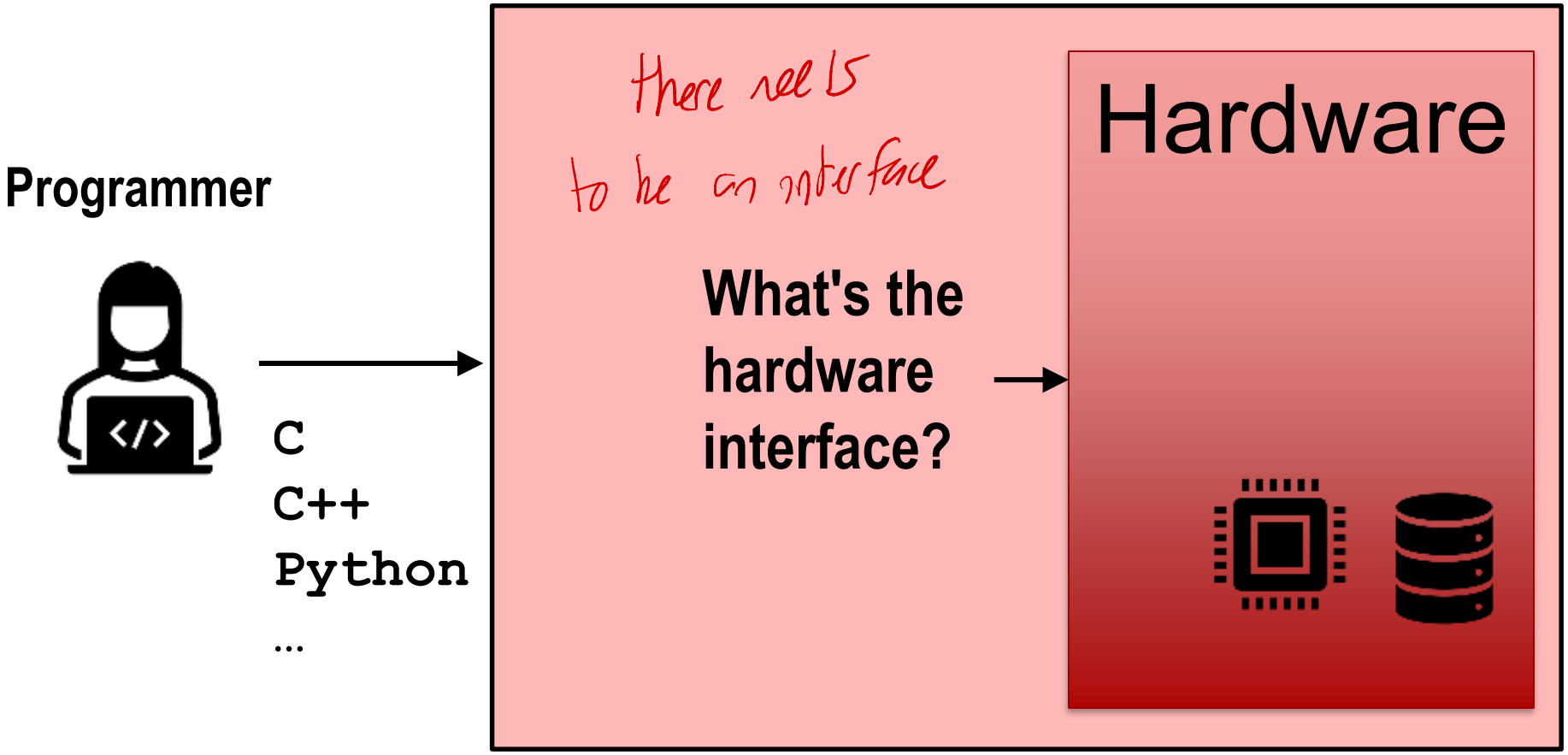
An object is: state + method

- **State is data**
- **Methods are operations on data** → manipulates the state of the object

These methods provide the *interface* of an object



Programming interface for users



Instruction Set Architecture (Architecture)

The architecture defines the

interface

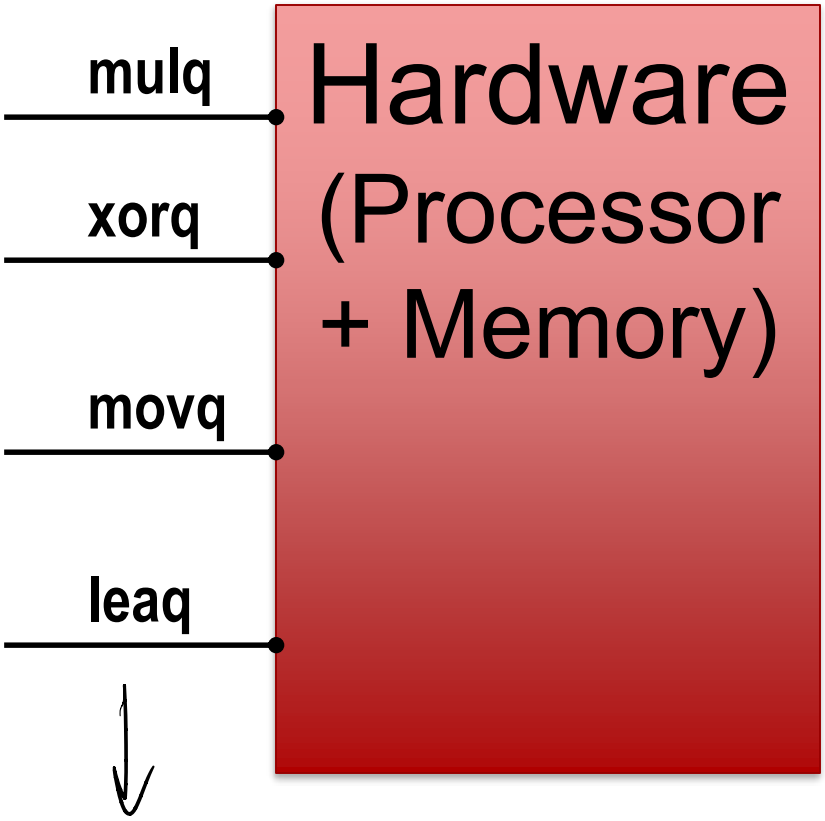
between software and hardware

An ISA defines (among others):

States in hardware – e.g., memory

(Note: there are different kinds of memory)

Methods: instructions used to change states (e.g., memory)



all of these manipulate the state in the hardware

Example

```
void swap (long *xp, long *yp)
{
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

```
swap:
    movq (%rdi), %rax
    movq (%rsi), %rdx
    movq %rdx, (%rdi)
    movq %rax, (%rsi)
    ret
```

Assembly code

these instructions are defined in the ISA

now we can mess with the hardware

mulq
xorq
movq
leaq

Hardware (Processor + Memory)

compiler's job compiles into most optimized assembly string

Compiling Into Assembly

C Code (sum.c)

```
long plus(long x, long y);

void sumstore(long x, long y,
              long *dest)
{
    long t = plus(x, y);
    *dest = t;
}
```

Generated x86-64 Assembly

```
sumstore:
    pushq   %rbx
    movq    %rdx, %rbx
    call    plus
    movq    %rax, (%rbx)
    popq    %rbx
    ret
```

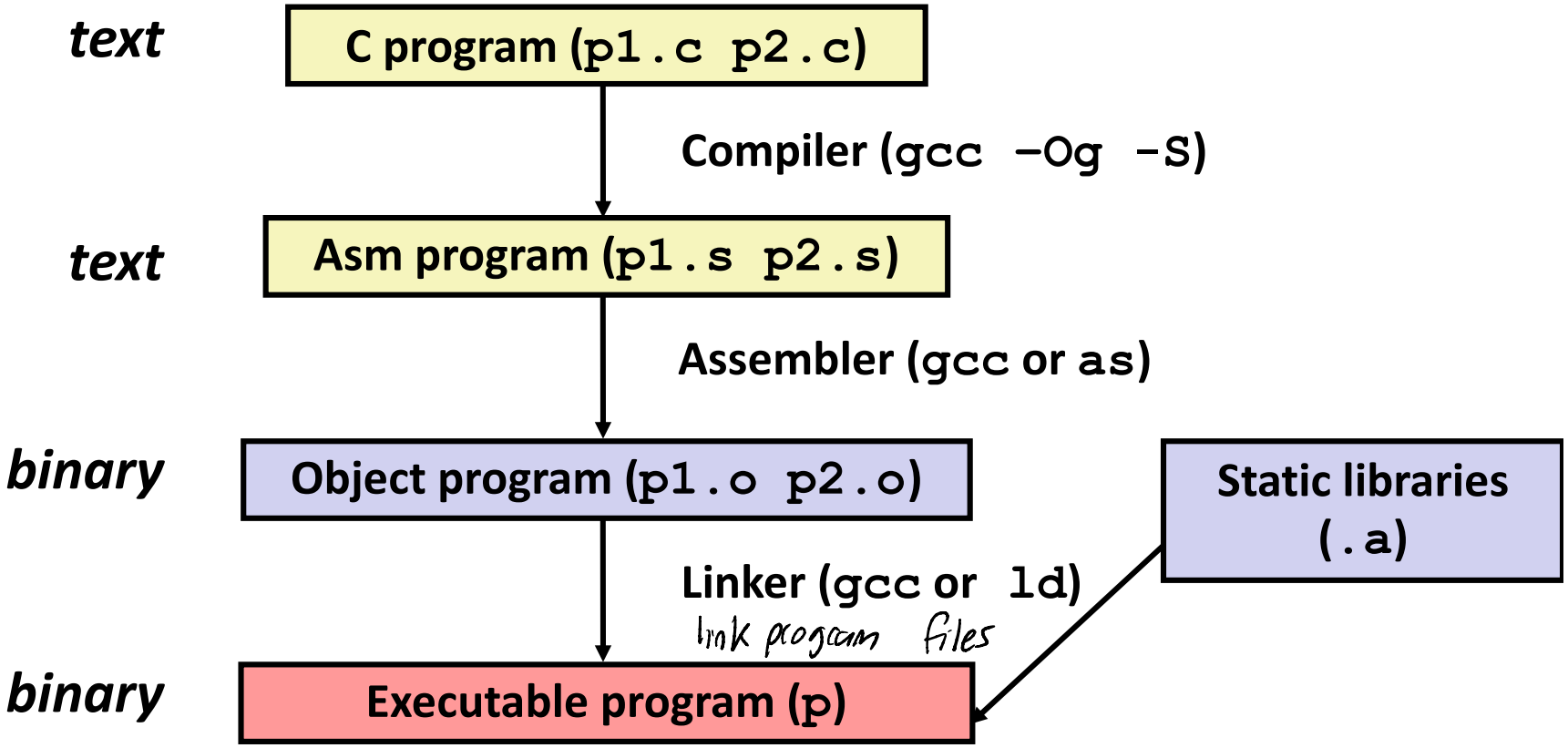
Example command

```
gcc -Og -S sum.c
```

Produces file `sum.s`

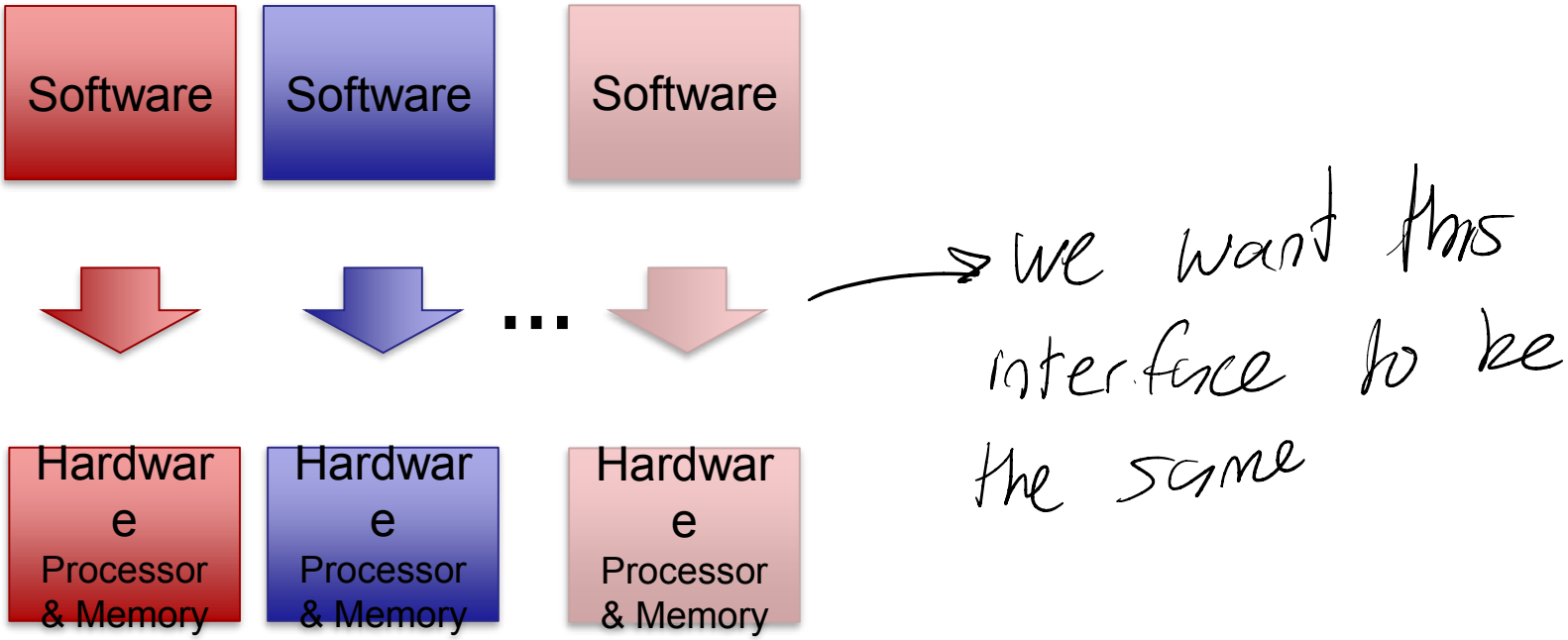
***Warning*: Will get very different results on different machines due to different versions of gcc and different compiler settings.**

Aside: Compiling



Why is ISA needed?

Early days of computer systems

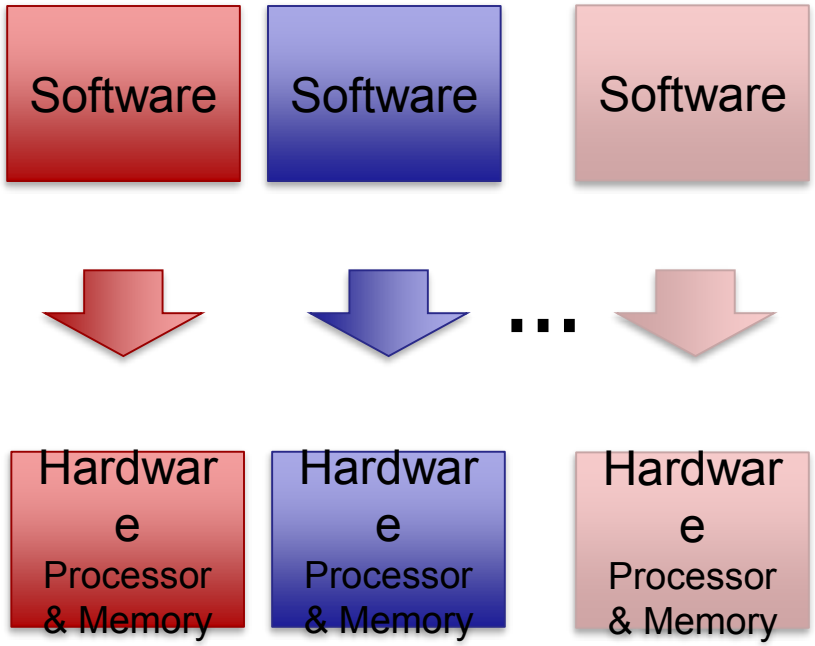


In 1960's, IBM had 4 incompatible lines of computers

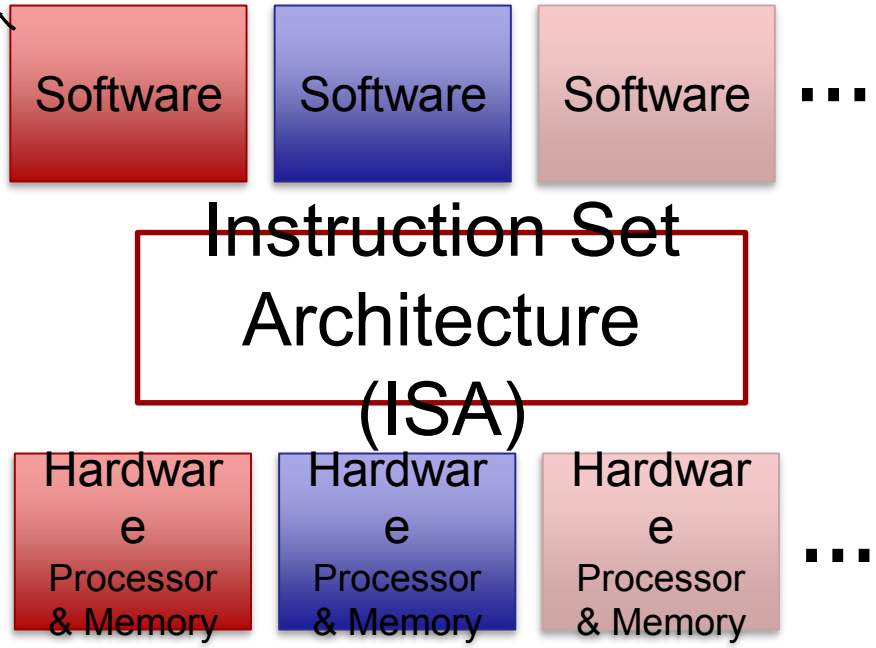
Why is ISA needed?

now the software doesn't care

Early days of computer systems



Faster innovation with ISA



Success of IBM 360

and the hardware just needs to follow rules

IBM 360: Initial Implementations

	Model 30	. . .	*Model 70*
Storage	8K - 64 KB		256K - 512 KB
Datapath	8-bit		64-bit
Circuit Delay	30 nsec/level		5 nsec/level

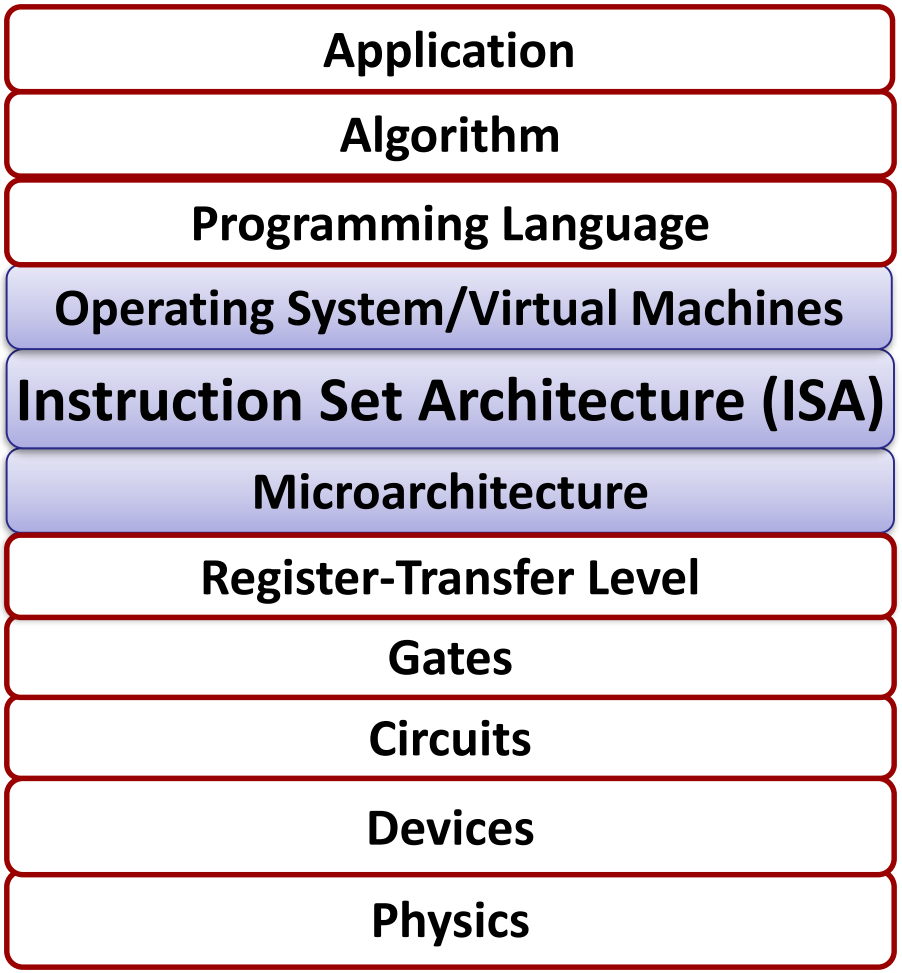
IBM 360 instruction set architecture (ISA) completely hid the underlying technological differences between various models.

Milestone: The first true ISA designed as portable hardware-software interface!

Designing an ISA

- **Architecture (interface) design is a tradeoff**
- **On the one hand**
  - Want a compatible line of processors
  - Don't want to rewrite all software when new architecture available
- **On the other hand**
  - Need to maintain interface; innovation tends to be slow

Big Picture

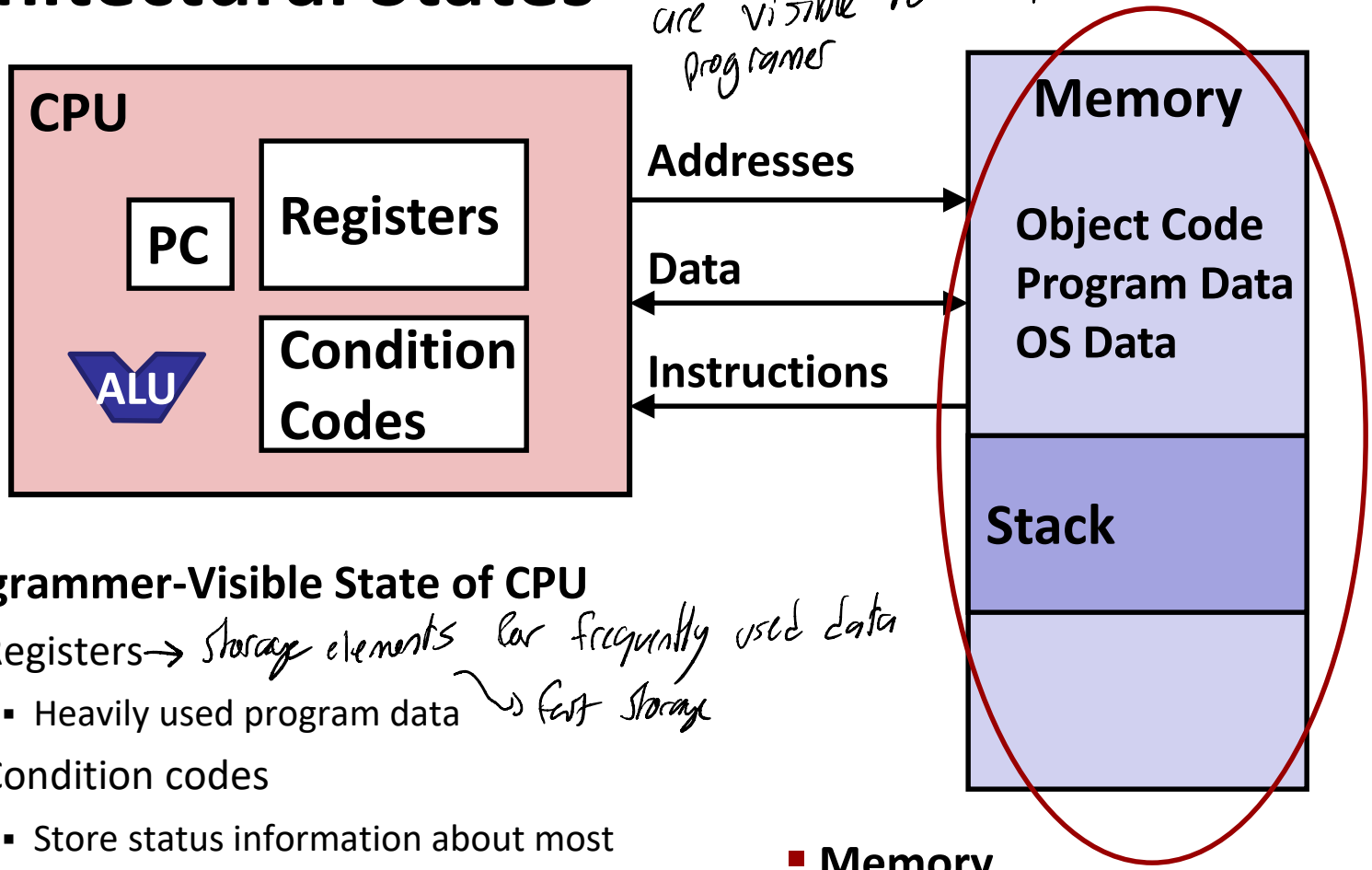


Lecture Goals

- Define Instruction Set Architecture (ISA) and motivate the need for ISA
- **Basics of memory address**
- Basics of x86-64 ISA

Architectural States

all of these are visible to programer and manipulatable



- **Programmer-Visible State of CPU**
  - Registers → storage elements for frequently used data → fast storage
    - Heavily used program data
  - Condition codes
    - Store status information about most recent arithmetic operation
    - Used for conditional branching
  - PC: Program counter → instruction pointer → where is the next instruction
    - Address of next instruction
    - Called "EIP" (IA32) or "RIP" (x86-64)
- **Memory**
  - Byte addressable array
  - Code, user data, (some) OS data
  - Includes stack used to support procedures

Memory basics

we can think of memory for now as a large array

- **Byte (8 bits) is (usually) the *smallest addressable unit in memory***
  - Every byte has a ***memory address***

- **Word is the nominal size of integer-valued data in a machine**
  - Word is also size of each ***memory address***
  - Word is also the size of a ***CPU register***

every memory location stores one byte of data.

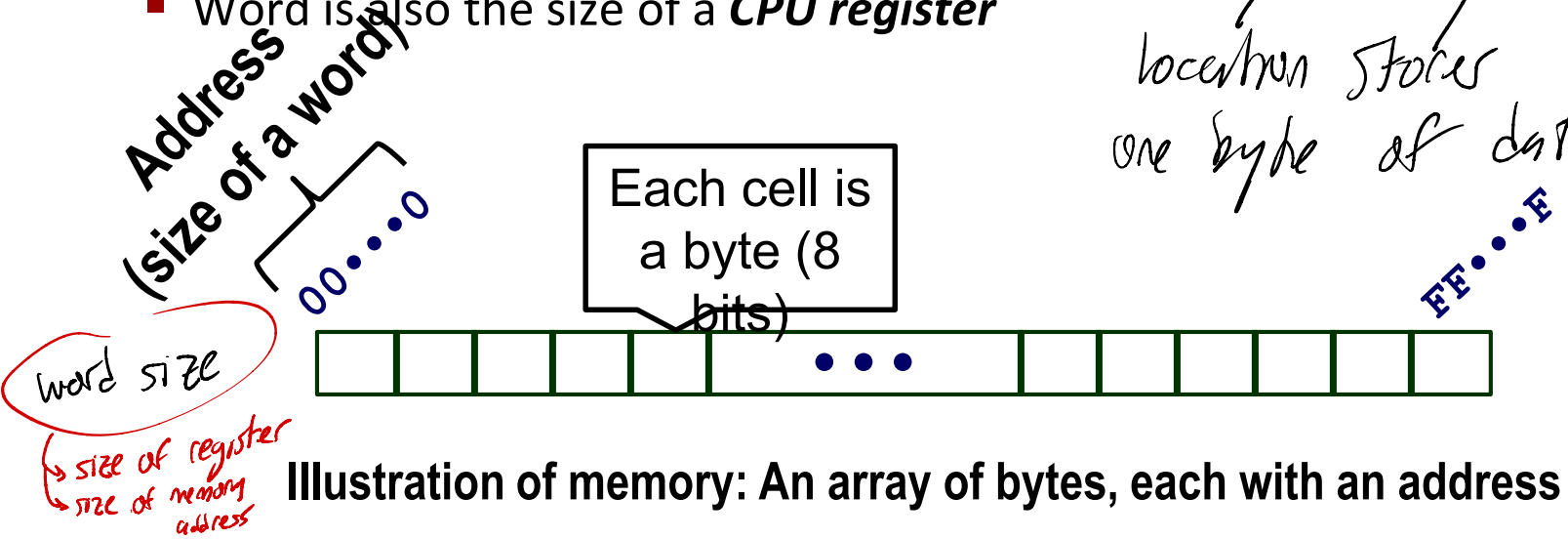


Illustration of memory: An array of bytes, each with an address

Bytes, Words, and Memory Size

- **Word size limits the memory size. Why?**
  size of memory-address
- **If a machine uses 8-bit words, what's the max memory size?**
  - 2^8 = 256 unique memory addresses ➔ max memory size = 256 B
    we can't have more distinct values → you can only point to 256 addresses because that's all your pointers
- **On 32-bit machines (32-bit words), what's the max memory size?**
  - 2^32 = 4GB (1G=1024M=2^30, 1M=1024K=2^10*2^10=2^20, 1K=1024=2^10)
  - Too small for memory-intensive jobs
- **These days: most machines are 64-bit, i.e., 64-bit words**
  - Potential address space ≈ 2^64 bytes ~= 1.8 X 10^19 bytes
  - x86-64 machines support 48-bit addresses: 256 Terabytes

Multi-Byte Variables & Byte Ordering

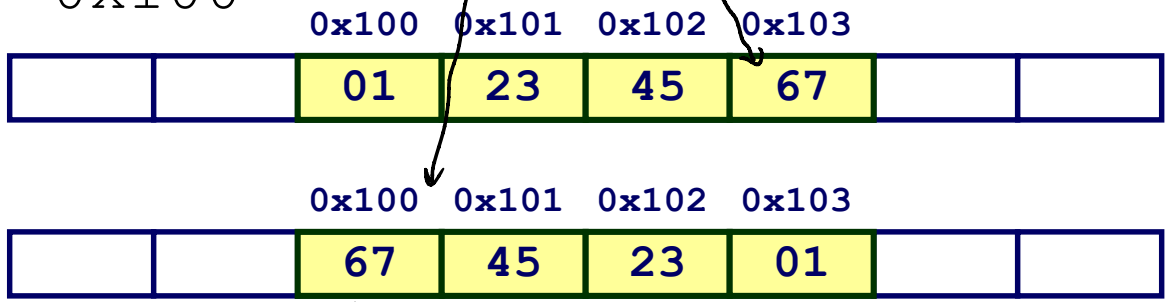
- **Multi-byte variables**
  - ex. short (2 bytes) int (4 bytes), long (8 bytes)
- **How are the bytes within a multi-byte word ordered in memory?**
- **Two conventions**
  - Big Endian: Least significant byte has highest address
  - Little Endian: Least significant byte has lowest address
    - x86, ARM processors running Android, iOS, Linux, and Windows
  - X endian: Least significant byte has X-est address

significance
biggest number in binary

4-byte variable: `x = 0x12345678`

Address: `&x = 0x100`

smallest in terms of number

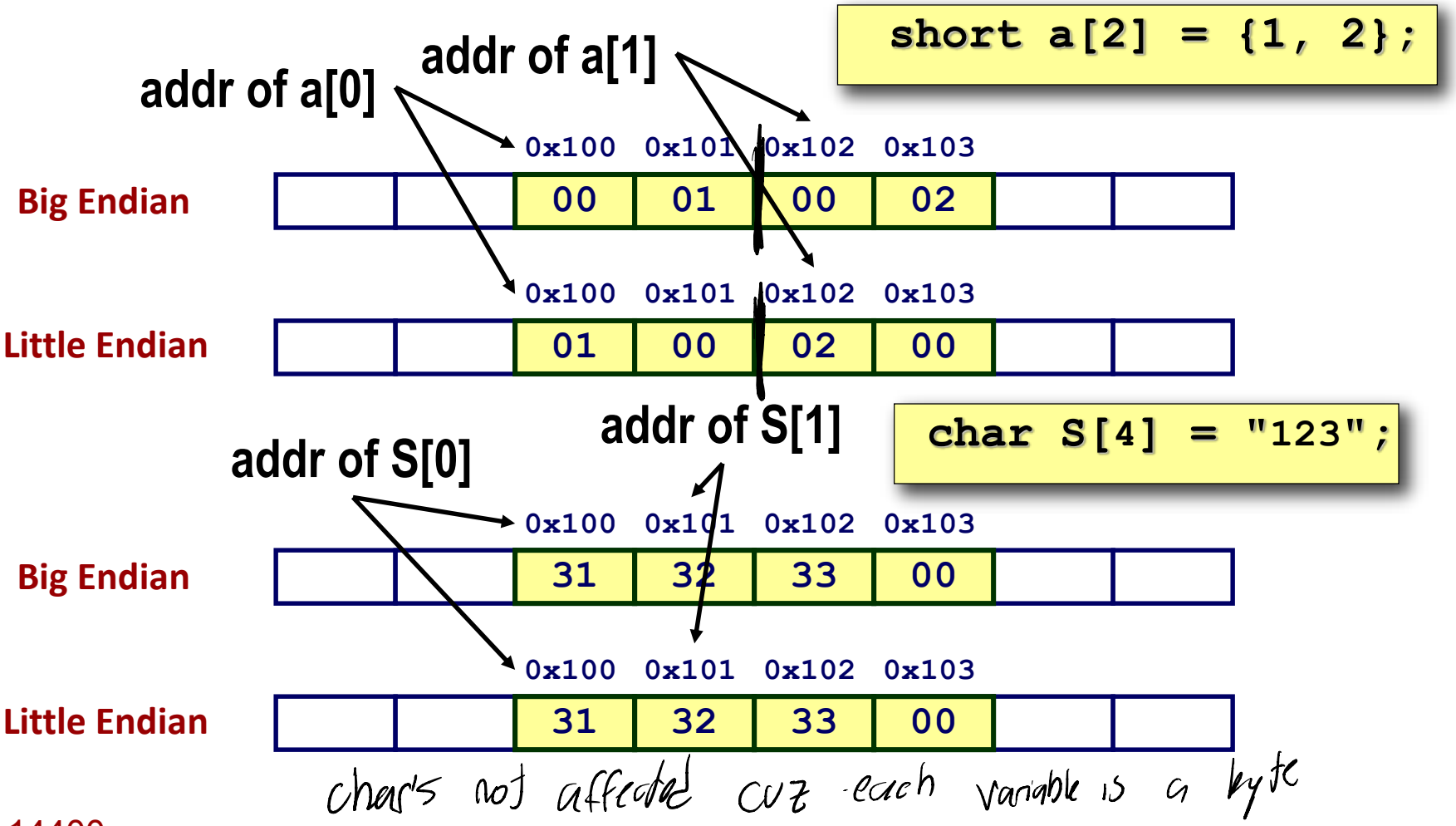


the more popular

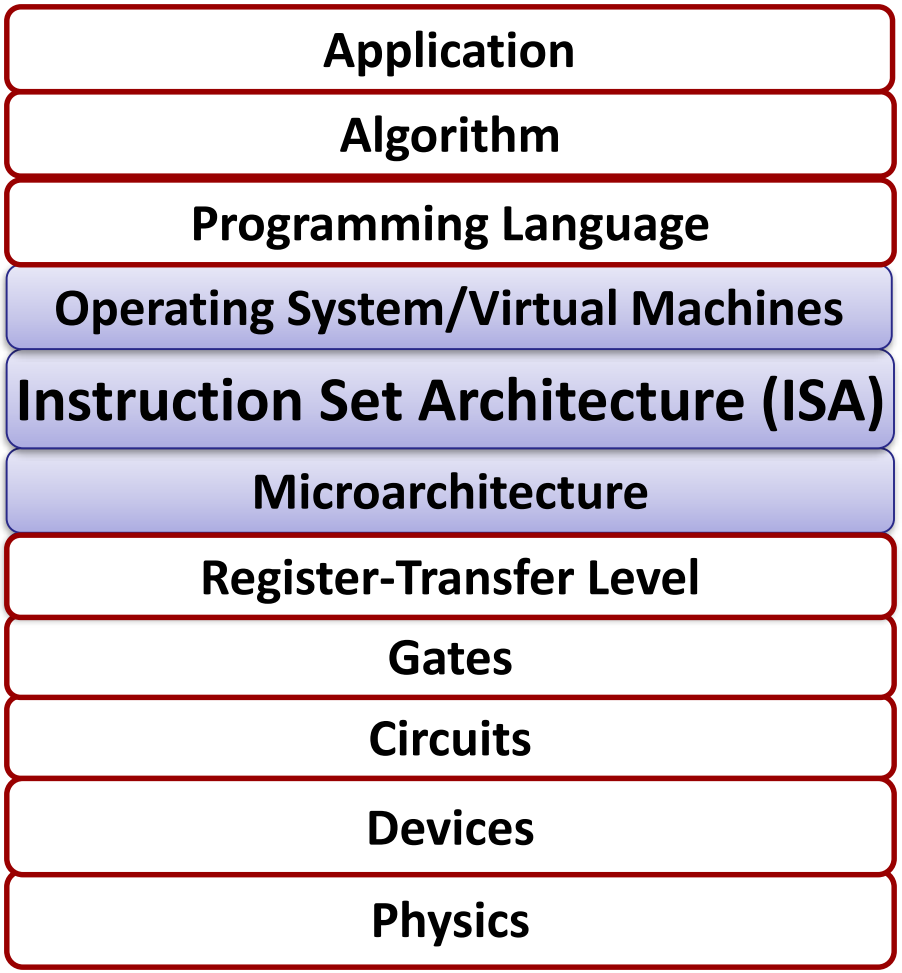
each a byte

Array memory layout

- Element addresses in an array are not affected by byte ordering



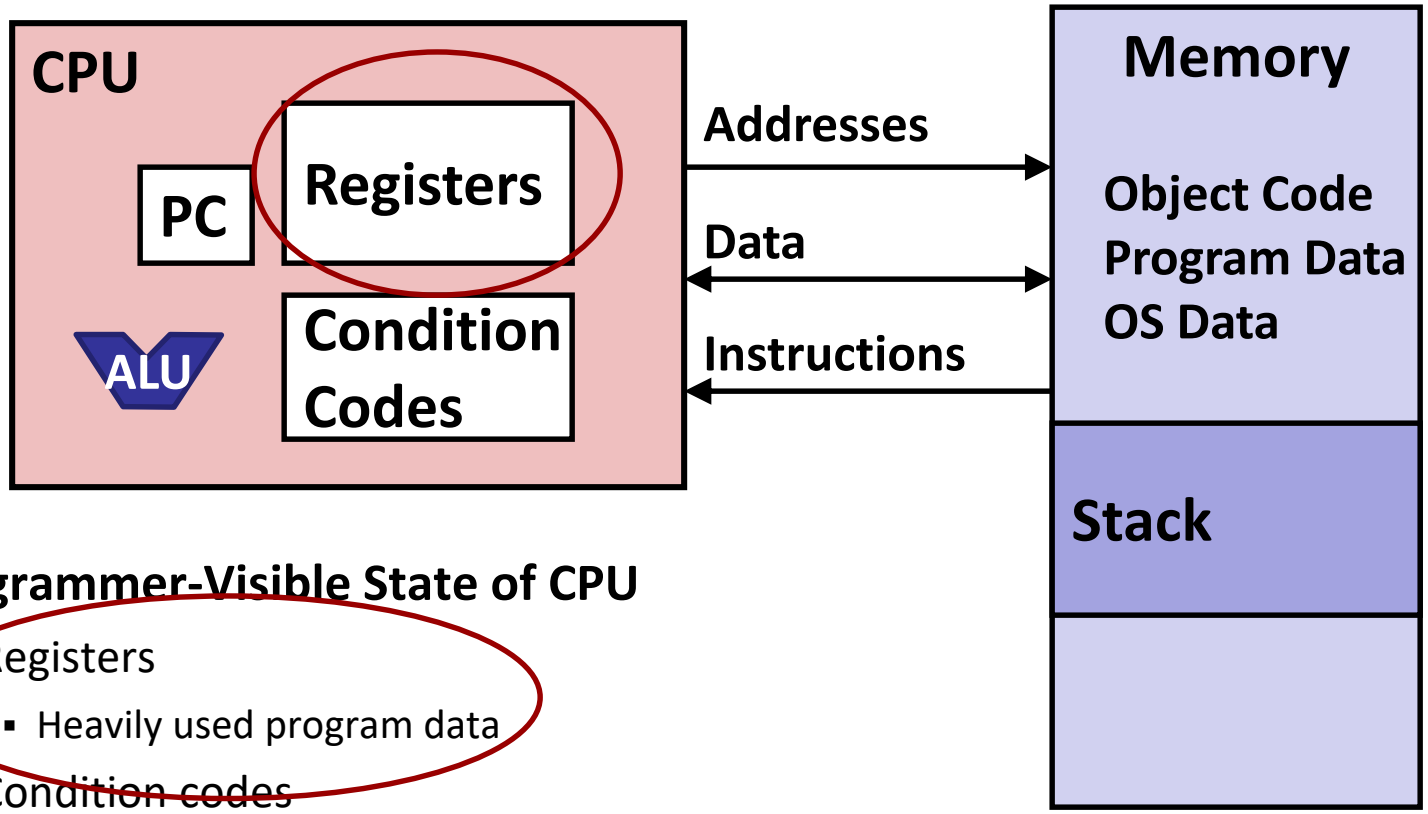
Big Picture



Lecture Goals

- Define Instruction Set Architecture (ISA)
- Basics of memory
- Basics of x86-64 ISA
  - **CPU Registers**

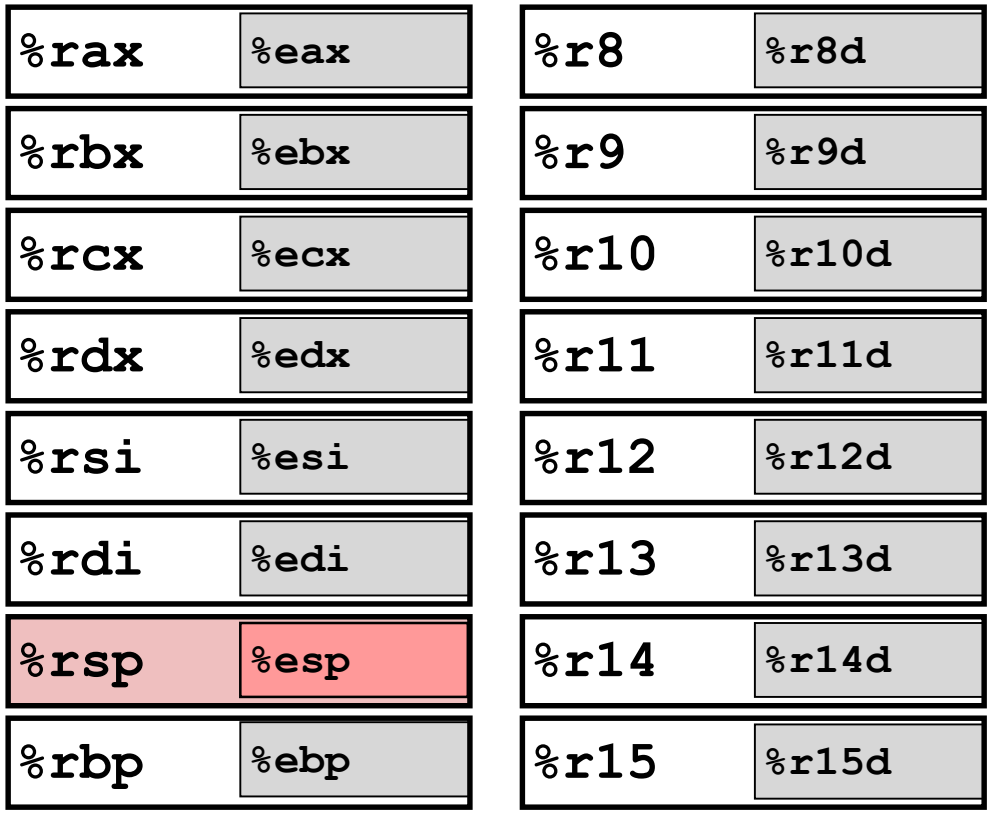
Architectural States



- **Programmer-Visible State of CPU**
  - Registers
    - Heavily used program data
  - Condition codes
    - Store status information about most recent arithmetic operation
    - Used for conditional branching
  - PC: Program counter
    - Address of next instruction
    - Called “EIP” (IA32) or “RIP” (x86-64)

- **Memory**
  - Byte addressable array
  - Code, user data, (some) OS data
  - Includes stack used to support procedures

x86-64 Integer Registers



- 64-bit long (word size)
  - Can reference low-order 4 bytes (also low-order 1 & 2 bytes)
- General-purpose registers (except %rsp)

registers have names that we call them by
even the lower part of the registers have names

Moving Data 101

- **Moving a constant value to a register**

```
movq $0x100, %rax
```

- **movq: instruction opcode**
  - the last character indicates the # of bytes to change/access
    - E.g., movq moves 8 bytes, movl moves 4 bytes, etc
- **%rax: destination**
- **$0x100: the value to be moved**
  - Prefix $ indicates it's a constant value

%rax	%r8
%rcx	%r9
%rdx	%r10
%rbx	%r11
%rsi	%r12
%rdi	%r13
%rsp	%r14
%rbp	%r15

Moving Data, in general

- **Moving Data**

  `movq` ***Source, Dest***

- **Operand Types**
  - **Immediate**: Constant integer data (explained in last slide)
    - Ex., movq $0x100, %rax
  - **Register**: One of 16 integer registers
    - Ex. movq %rax, %r13
    - Copy the value in %rax to %r13
  - **Memory**
    - Ex. movq (%rax), %r13
    - Use the value in %rax as a memory address and copy the value from memory to %r13.

`%rax`	`%r8`
`%rcx`	`%r9`
`%rdx`	`%r10`
`%rbx`	`%r11`
`%rsi`	`%r12`
`%rdi`	`%r13`
`%rsp`	`%r14`
`%rbp`	`%r15`

Assembly 101

```
void swap
    (long *xp, long *yp)
{
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

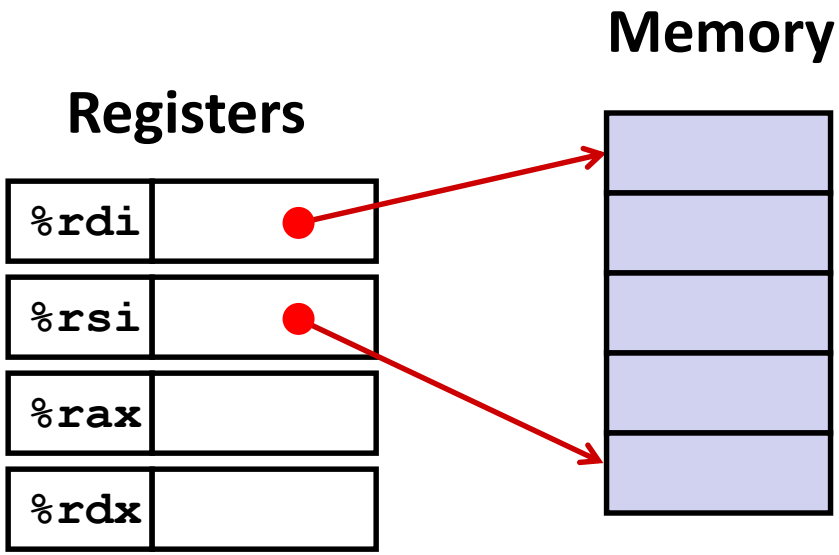
```
swap:
    movq    (%rdi), %rax
    movq    (%rsi), %rdx
    movq    %rdx, (%rdi)
    movq    %rax, (%rsi)
    ret
```

Understanding `Swap()`

```
void swap
    (long *xp, long *yp)
{
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1



Understanding `Swap()`

Registers

%rdi	0x120
%rsi	0x100
%rax	
%rdx	

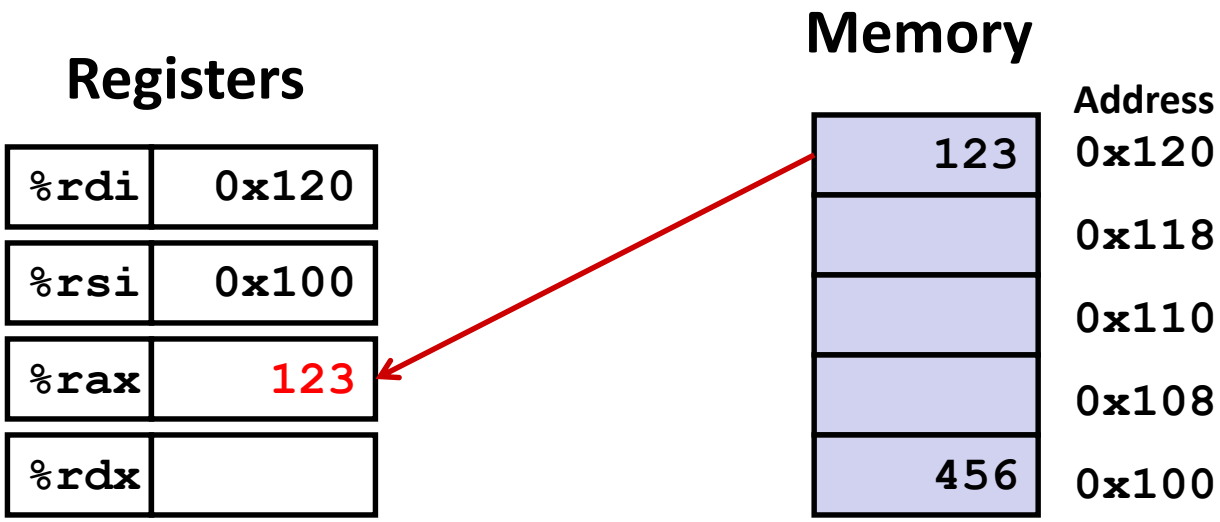
Memory

	Address
123	0x120
	0x118
	0x110
	0x108
456	0x100

Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

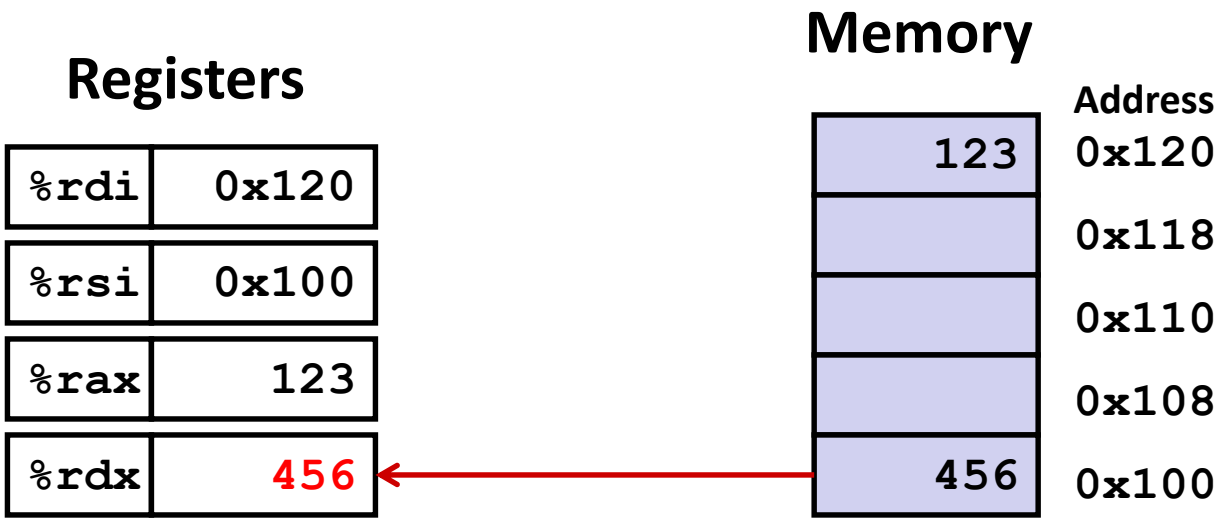
Understanding `Swap()`



Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

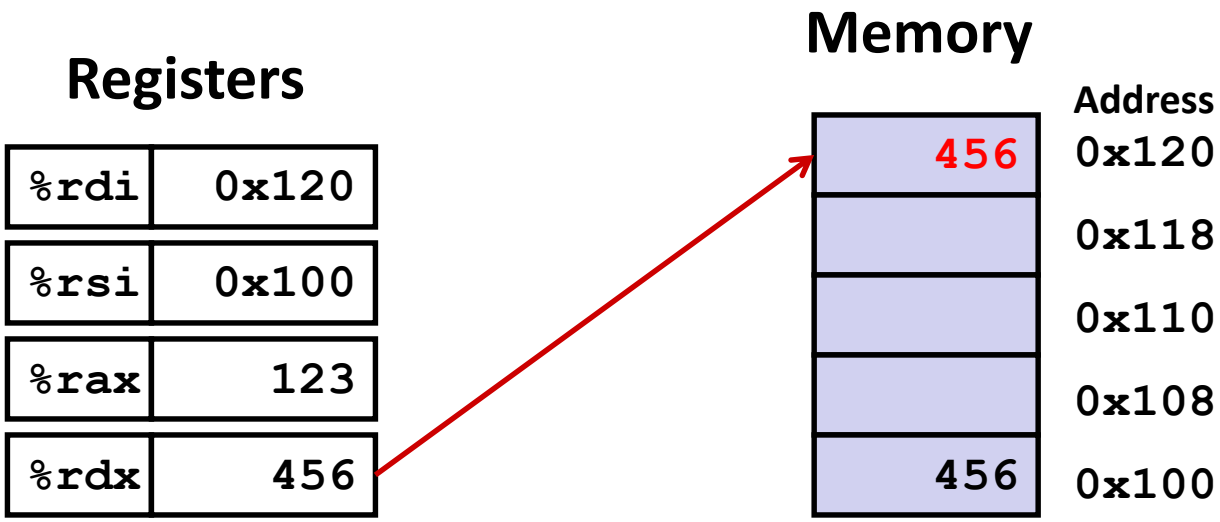
Understanding `Swap()`



Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

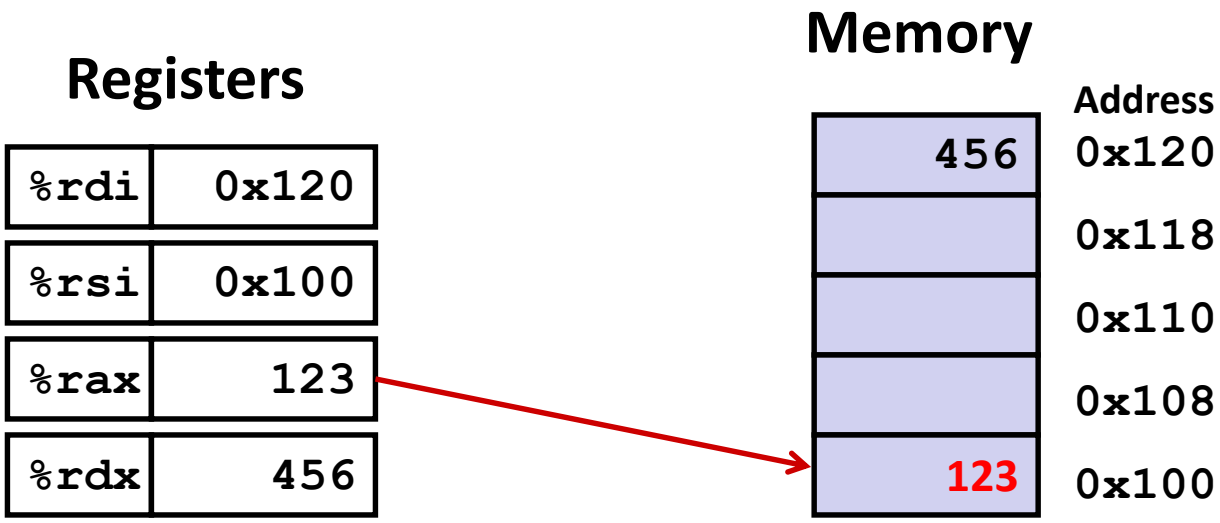
Understanding `Swap()`



Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

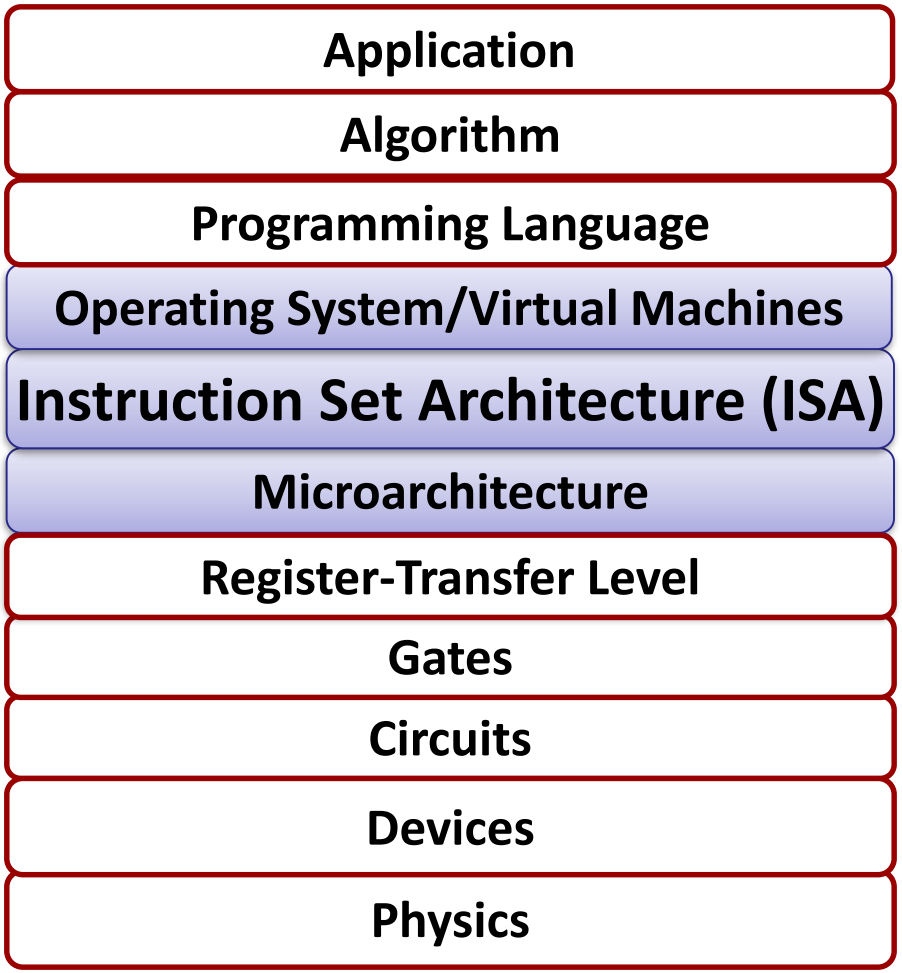
Understanding `Swap()`



Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

Big Picture



Lecture Goals

- Define Instruction Set Architecture (ISA)
- Basics of memory
- Basics of x86-64 ISA
  - CPU Registers
- Next lecture: Registers and Memory